OUTAGE DISSECTIONS • OPERATIONAL RELIABILITY • HIGH AVAILABILITY

Dissecting GitHub Outage: Blind Spots in "Guaranteed" High Availability

Curated from Arpit Bhayani's Architecture Dissection Series • Core Focus: Cold Caches & Thread Saturation

EXECUTIVE ARCHITECTURAL ABSTRACT

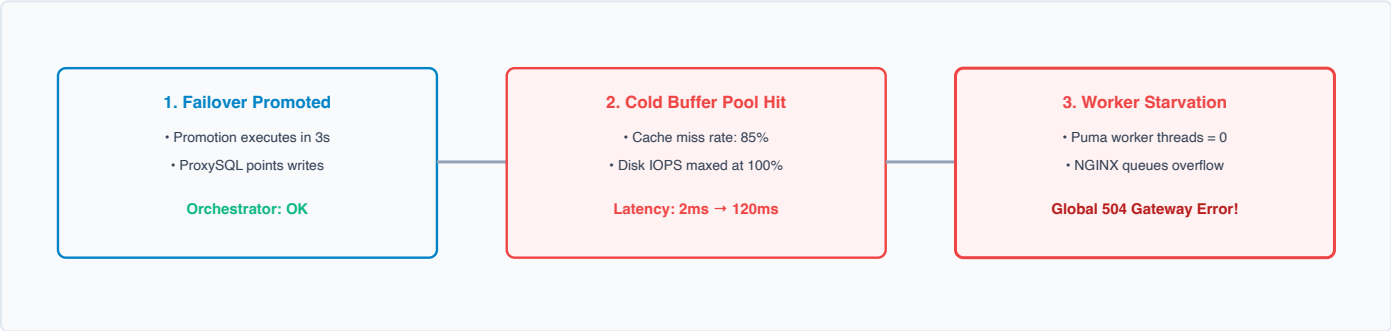
High availability cannot be validated solely by database health checks. In this critical operational incident, automated failover successfully promoted a standby MySQL replica in seconds. However, because the newly promoted replica possessed a **Cold InnoDB Buffer Pool**, disk seek bottlenecks caused query response times to surge from 2ms to 120ms. In accordance with **Little's Law**, this 50x latency increase caused rapid exhaustion of upstream application worker pools, triggering global HTTP 504 timeouts.

1. THE DISCONNECT BETWEEN DATABASE FAILOVER & USER AVAILABILITY

Automated database failover often fails silently from the application perspective:

- **Database Health Check Deception:** Monitoring reported the database was 100% healthy because MySQL was accepting connections and running queries.
- **Physical Cache Thrashing:** The promoted node had only read-query working sets cached in RAM. Incoming write transactions forced synchronous disk reads, pegging NVMe IOPS to 100%.
- **Cascading Web Tier Collapse:** In-flight requests held Rails worker processes 50x longer than usual, starving reverse proxies and triggering gateway collapses.

2. VISUALIZING THE COLD CACHE CASCADE



3. LITTLE'S LAW: THE MATHEMATICAL PROOF OF WORKER STARVATION

The mathematical certainty of application collapse under database latency spikes is proven by Little's Law:

Mathematical Formulation: $L = \lambda \times W$

Let λ = 10,000 req/sec (arrival rate), W = average service duration, and L = required concurrent in-flight worker processes.

- **Warm Steady State (W = 4ms = 0.004s):**
  `L_normal = 10,000 × 0.004 = 40 concurrent workers`. The web fleet operates with massive spare capacity.
- **Cold Cache State (W = 120ms = 0.120s):**
  `L_cold = 10,000 × 0.120 = 1,200 concurrent workers`.

Because the web tier is provisioned for 400 concurrent worker threads, the capacity is exceeded by 300%. Within 150ms, all worker threads are locked in `wait_for_query`, halting request ingestion.

4. ENGINEERING REMEDIATION: BUFFER POOL SYNCHRONIZATION

Modern architectures eliminate cold failover shock through active cache warming and throttled cutovers:

1. Periodic Buffer Pool Dumps

Replicas maintain identical memory footprints by continuously sampling the master's buffer pool table. Upon promotion, hot index pages are preloaded into RAM before live write traffic is redirected.

2. ProxySQL Progressive Ramping

ProxySQL ramps traffic in stages: 20% on second 1, 50% on second 15, and 100% at second 45. Shedding excess write traffic during the first 30 seconds prevents worker thread pool exhaustion.

5. CONFIGURATION BLUEPRINT FOR BUFFER POOL PERSISTENCE

```
# my.cnf configuration on all cluster nodes
[mysqld]
# Continuously dump buffer pool state
innodb_buffer_pool_dump_at_shutdown = ON
innodb_buffer_pool_dump_pct = 80

# Replicas trigger warm dumps every 10 minutes
innodb_buffer_pool_dump_now = ON

# Automatically preload buffer pool pages on startup/promotion
innodb_buffer_pool_load_at_startup = ON
innodb_buffer_pool_load_now = ON
```

6. ARCHITECTURE RESILIENCE COMPARISON

Failover Architecture	Failover Duration	Post-Promotion Latency	Thread Starvation Protection
Naive Master Failover	3–5 seconds	50–200ms (High Disk I/O)	**None (Triggers HTTP 504 blackouts)**
Pre-Warmed Replicas	3–5 seconds	3–6ms (Near-RAM speeds)	**High (Buffer pool contains hot set)**
Adaptive Load Shedding (ProxySQL)	5–10 seconds	Bounded to 10ms	**Complete (Fails non-critical writes fast)**

7. PRODUCTION ENGINEERING TAKEAWAYS

1. **Measure End-to-End Availability, Not Node Up-time:** An automated failover that completes without error while application error rates spike to 100% is an operational failure. Define SLIs/SLOs at the ingress boundary.
2. **Always Account for Little's Law in Capacity Planning:** A temporary 10x latency spike requires 10x more concurrency. Systems that lack concurrency limiters and load-shedding mechanisms will always collapse under latency expansion.
3. **Cold Caches are Systemic Threats:** Whether in database buffer pools, Redis clusters, or microservice local caches, promoting or booting cold instances during peak traffic invites immediate outage cascades.